



Universidad Nacional Autónoma de México

Facultad de Ciencias

Práctica Número 4

Cristian Guillermo Mejia Cisneros

Número de cuenta: 427329650

24 Septiembre de 2026

1. Objetivos

- **Variables y tipos de datos:** Declarar variables en Java, asignarles valores y entender el tipo que se les asigna determina que se puede guardar en ellas.
- **Comentarios:** Usar los distintos tipos de comentarios de Java y comprobar que el compilador los ignora por completo.
- **Formato de salida:** Investigar herramientas para presentar números y textos de forma legible.
- **Abstracción:** Leer un problema planteado en español y obtener la información para escribir el código.

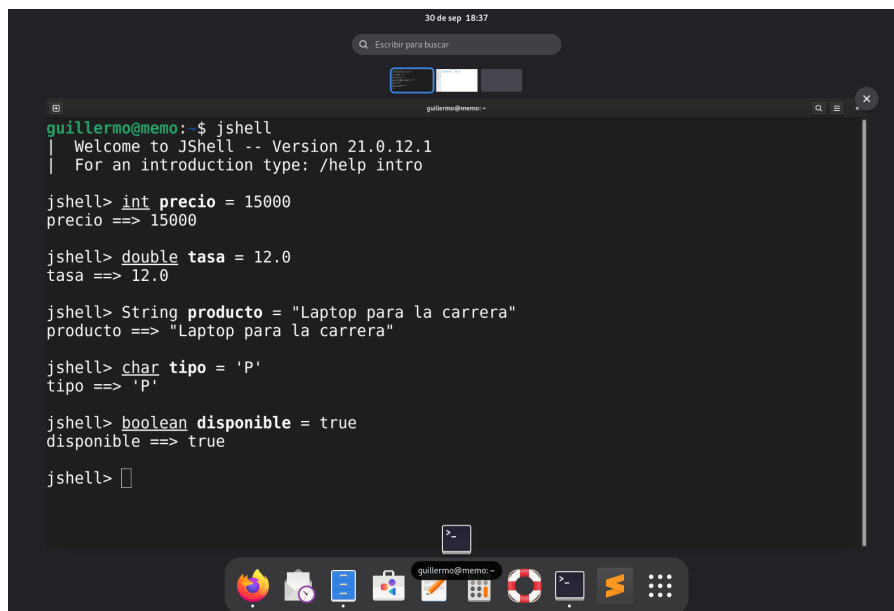
2. Desarrollo

2.1. Parte 1: Variables

2.1.1. Actividad 1.1 - Variables en Jshell

1. Para comenzar está práctica, observemos la **Figura 1**, en la que hemos ejecutado los siguientes comandos en Jshell:

```
1 int precio = 15000
2 double tasa = 12.0
3 String producto = "Laptop para la carrera"
4 char tipo = 'P'
5 boolean disponible = true
```



The screenshot shows a terminal window titled 'guillermo@memo: -' with a search bar at the top. The terminal output is as follows:

```
guillermo@memo:~$ jshell
Welcome to JShell -- Version 21.0.12.1
For an introduction type: /help intro

jshell> int precio = 15000
precio ==> 15000

jshell> double tasa = 12.0
tasa ==> 12.0

jshell> String producto = "Laptop para la carrera"
producto ==> "Laptop para la carrera"

jshell> char tipo = 'P'
tipo ==> 'P'

jshell> boolean disponible = true
disponible ==> true

jshell> 
```

Figura 1: *Declaraciones realizadas en Jshell.*

2. De nuevo dentro de Jshell, escribimos ahora solamente el nombre de las variables, donde obtendremos el valor antes declarado como podemos observar en la **Figura 2**.

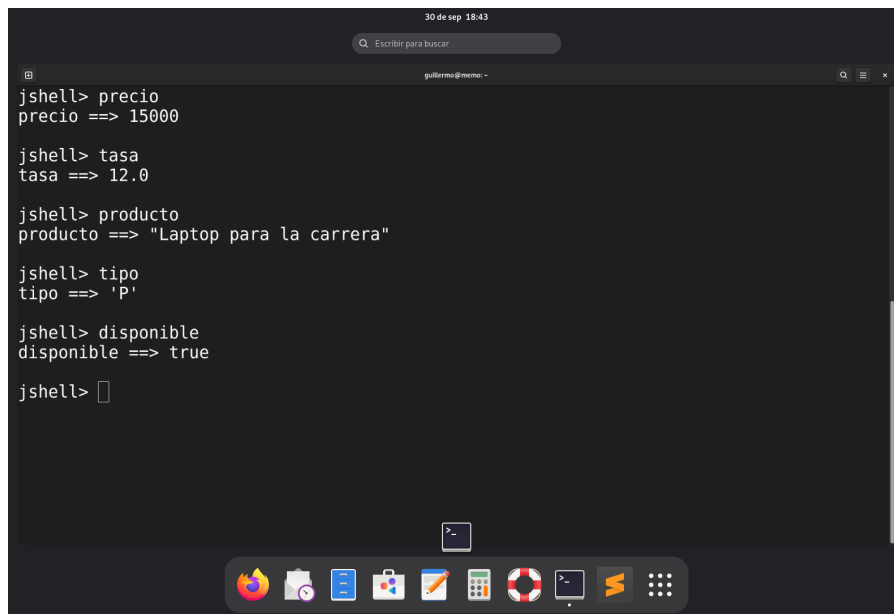


Figura 2: *Ejecución de variables previamente declaradas.*

3. Ahora si ejecutamos lo siguiente en Jshell:

```
1 precio
2 precio = 18000
3 precio
```

Tendremos los resultados de la **Figura 3**, notando que en realidad, si le asignamos otro valor a la variable **precio**, este cambiara el número que tenia guardado anteriormente.

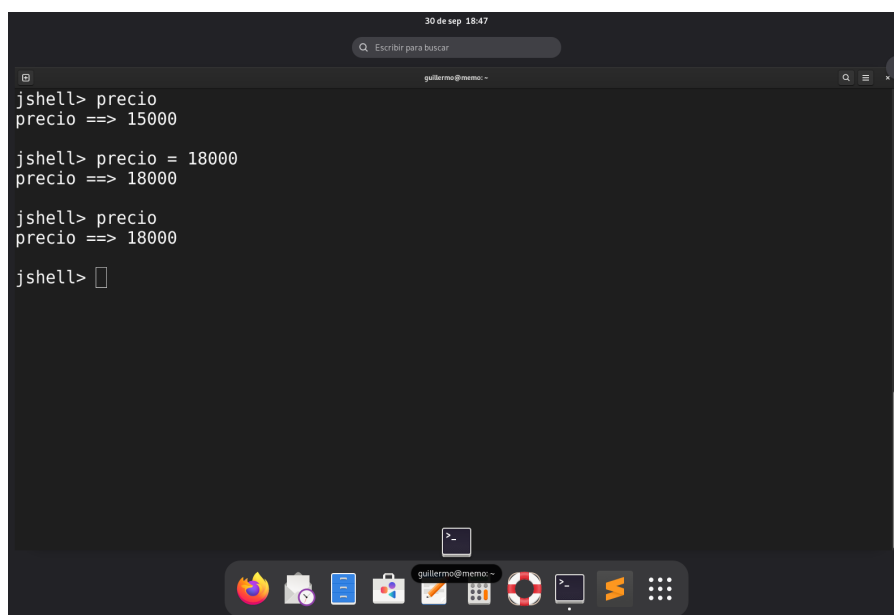


Figura 3: *Cambio del valor de la variable **precio**.*

4. Vamos a realizar las siguientes operaciones con las variables **precio**, **tasa** y **producto**:

```
1 precio*0.84
2
3 tasa + 4.0
```

4
5

```
producto + " CC"
```

Las cuales son operaciones totalmente válidas, en el caso de **precio**, lo multiplicamos por 0,84 para obtener el precio sin IVA, y opera sin problemas ya que al multiplicar un **int** con un **double**, este último es más grande por lo que predomina y el resultado terminara siendo un **double**.

Para el caso de **tasa**, solo estamos haciendo cálculos entre valores tipo **double**.

Por último, en el caso de **producto**, simplemente el símbolo +, está concatenando el texto de la variable **producto** con la cadena de caracteres " CC". Así es como obtenemos el resultado de la **Figura 4**.



```
jshell> precio*0.84
$20 ==> 15120.0

jshell> tasa + 4.0
$21 ==> 16.0

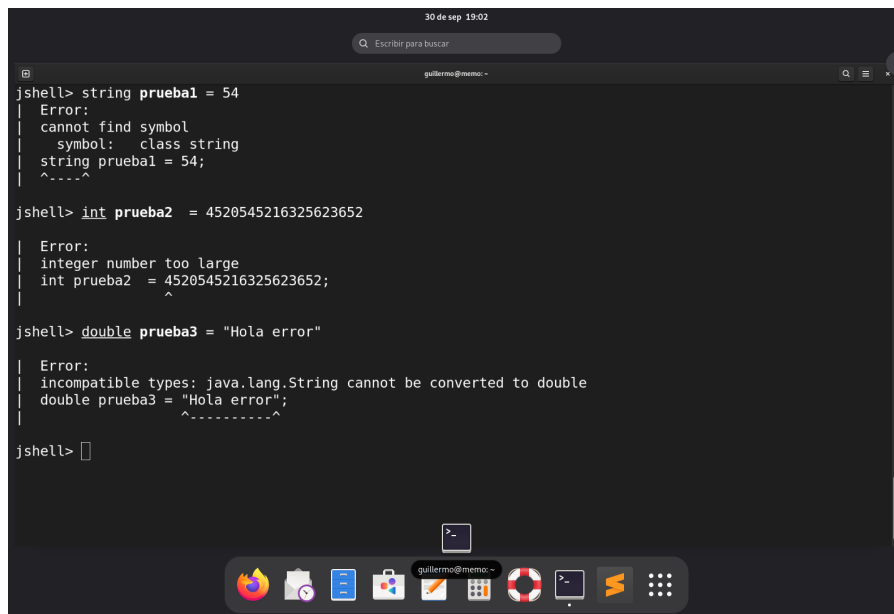
jshell> producto + " CC"
$22 ==> "Laptop para la carrera CC"

jshell>
```

Figura 4: Usando las variables declaradas para realizar operaciones.

5. Ahora vamos a declarar variables, de tal forma que les asignemos valores totalmente erróneos, es decir, que no sean del tipo que corresponden, en la **Figura 5** podremos ver las declaraciones de las variables **pruebaa1**, **prueba2** y **prueba3** (junto con sus respectivos errores) para desarrollar esta parte de la práctica. Los comandos introducidos en Jshell son los siguientes:

```
1 string prueba1 =54
2
3 int prueba 2 = 4520545216325623652
4
5 double prueba3 = "Hola error"
```



```
jshell> string prueba1 = 54
| Error:
| cannot find symbol
|   symbol:   class string
|   string prueba1 = 54;
|   ^^^^^^

jshell> int prueba2 = 4520545216325623652
| Error:
| integer number too large
| int prueba2 = 4520545216325623652;
|               ^

jshell> double prueba3 = "Hola error"
| Error:
| incompatible types: java.lang.String cannot be converted to double
| double prueba3 = "Hola error";
|                   ^^^^^^^^^^^

jshell> 
```

Figura 5: *Declaración de variables erróneas.*

Para cada una de las declaraciones erróneas, obtenemos un error distinto:

- `string prueba 1 = 54`: El error que obtuvimos, el cual podemos observar en la **Figura 5**, según entendemos, es porque Jshell, al realizar una asignación a una variable tipo **String**, espera recibir un símbolo, texto, el cual siempre colocamos entre comillas (“ ”), por lo que al solo colocar 54, Jshell no puede guardar ese valor en un **String**.
- `int prueba2 = 4520545216325623652`: El tipo de valor **int** solo puede guardar hasta cierta cantidad de dígitos o cierto valor de número (en particular desde $-2,147,483,648$ a $2,147,483,647$), el valor que tratamos de asignarle se sale del rango, por lo que Jshell no lo puede asignar, que es justo lo que el error marcado en el Jshell trata de decirnos (véase **Figura 5**).
- `double prueba3 = "Hola error"`: Para este caso, el error que Jshell regresa (**Figura 5**), indica que no podemos guardar un valor tipo **String**, en un **double**.

2.1.2. Actividad 1.2 - Agregando variables en P2

1. Para esta sección, hemos reutilizado el archivo `Bienvenida.java` de la práctica 2, para hacerle un par de modificaciones. Primero que nada guardamos el archivo, lo guardamos en el directorio `.../Repositorio/Practica04` y posteriormente lo abrimos con nuestro editor de texto Sublime Text.
2. Ahora, en vez de imprimir los datos como anteriormente lo habíamos hecho:

```
1 public class Bienvenida {
2     public static void main(String[] args){
3
4         System.out.println("Cristian Guillermo Mejia Cisneros\
5                             n427329650\nDoom Eternal");
6     }
7 }
```

Ahora vamos a declarar tres variables que contengan la información del nombre, número de cuenta y tema de interés, donde tenemos lo siguiente:

```
1      public class Bienvenida {
2      public static void main(String[] args){
3
4          int numeroCuenta = 427329650;
5          String miNombre = "Cristian Guillermo Mejia Cisneros";
6          String temaDeInteres = "Doom Eternal";
7
8          System.out.println(miNombre + "\n" + numeroCuenta + "\n" +
9              temaDeInteres);
10     }
```

El cual compila correctamente y ejecuta lo esperado, hecho que se puede ver en la **Figura 6**.

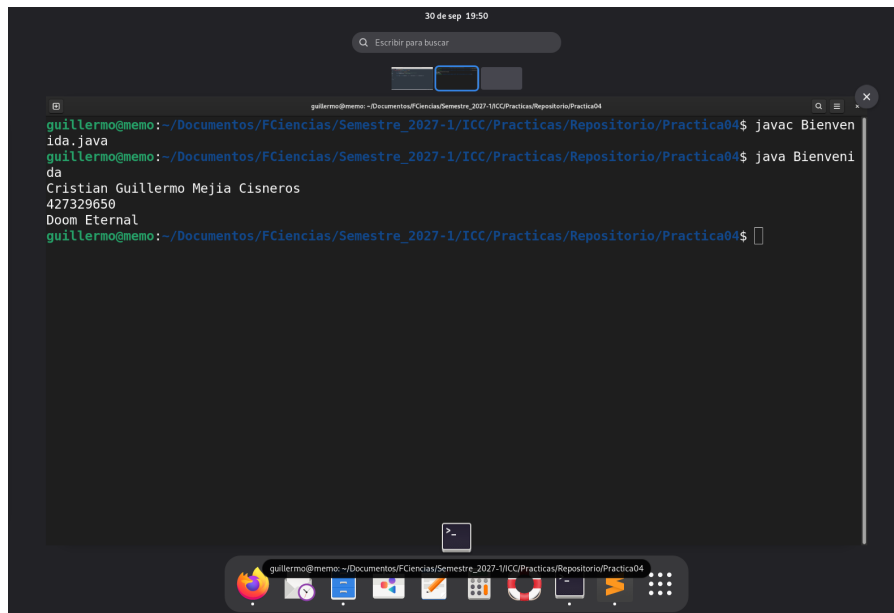


Figura 6: *Compilación y ejecución de Bienvenida.java* Con la información guardada en variables.

3. Posteriormente, se nos pide crear dos nuevas variables con nuestra edad e inicial de nuestro nombre o apodo favorito, para este caso utilizamos la edad 26 y el apodo **Memo**, obteniendo lo siguiente:

```
1      public class Bienvenida {
2      public static void main(String[] args){
3
4          int numeroCuenta = 427329650;
5          String miNombre = "Cristian Guillermo Mejia Cisneros";
6          String temaDeInteres = "Doom Eternal";
7          int edad = 26;
8          char inicialApodo = 'M';
9
10         System.out.println(miNombre + "\n" + numeroCuenta + "\n" +
11             temaDeInteres);
```

```

11     System.out.println(edad + "\n" + inicialApodo);
12
13     }
14 }

```

Lo anterior compila y ejecuta sin problema como podemos verlo en la **Figura 7**.

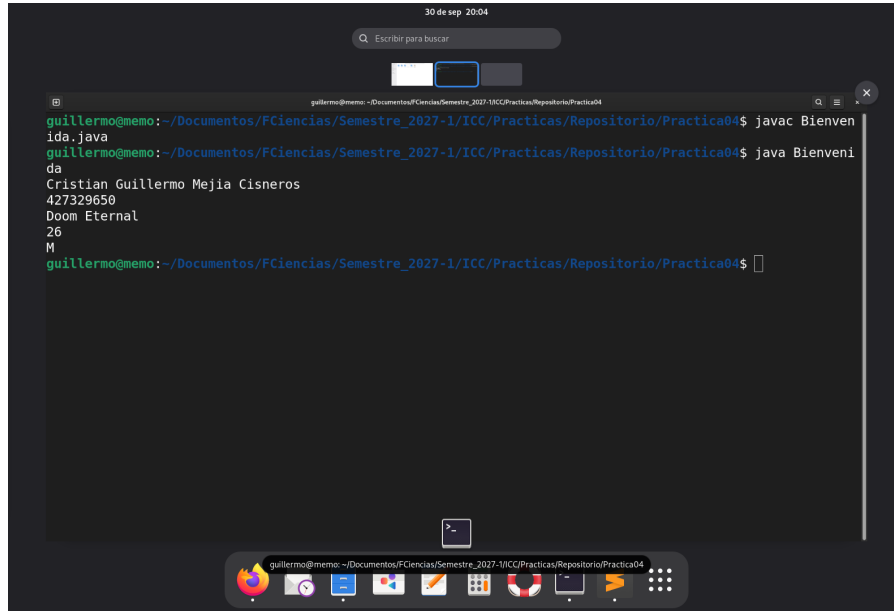


Figura 7: *Compilación y ejecución de Bienvenida.java* Con las nuevas variables agregadas.

2.2. Parte 2: Comentarios e impresiones elegantes.

2.2.1. Actividad 2.1: Comentario Java

1. Para este punto del reporte, vamos a descargar el archivo Programa.java que es una versión modificada de nuestro archivo Errores.java de la practica 2. En este archivo tenemos código al cual, por cada linea, debemos agregar un comentario que explique la utilidad de esta misma. En nuestro caso escribimos lo siguiente:

```

1      /*Declaracion de clase publica Programa, es el contenedor que
2      tiene adentro las variables y metodos que haran funcionar
3      al programa
4
5      */
6      public class Programa{
7
8      /* metodo main, importante ya que la maquina virtual de Java (JVM
9      ) empezara a leer todo lo que hemos programado, si este metodo
10     falta, aunque el programa se compila sin problema, no se
11     ejecutara de manera correcta*/
12     public static void main(String[] args) {
13
14     /*Declaracion de la variable producto, describe de que producto
15     estamos hablando*/
16     String producto = "Laptop para la carrera";

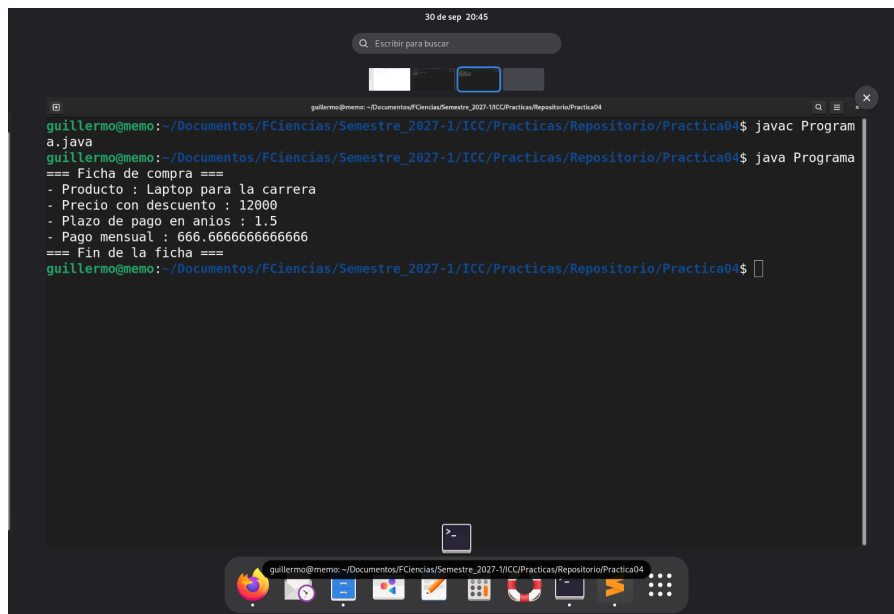
```

```

11  /*Declaracion de la variable precio, describe el precio del
    producto*/
12      int precio = 15000;
13
14  /*Declaracion de la variable descuento, describe el descuento
    hecho al producto*/
15      int descuento = 3000;
16
17  /*Declaracion de la variable meses, describe los meses en interes
    en que se puede pagar el producto*/
18      double meses = 18.0;
19
20  //Imprime "=== ficha de compra ==="
21      System.out.println("=== Ficha de compra ===");
22
23  // imprime la variable producto
24      System.out.println("- Producto : " + producto);
25
26  /*realiza la operacion de descuento al producto e imprime el
    resultado*/
27      System.out.println("- Precio con descuento : " + (precio -
        descuento));
28
29  //Imprime el plazo en anios, calculandolo primero
30      System.out.println("- Plazo de pago en anios : " + (meses /
        12.0));
31
32  //Imprime pago en meses, calculandolo primero
33      System.out.println("- Pago mensual : " + ((precio - descuento)
        / meses));
34
35  //Imprime "=== Fin de la ficha ==="
36      System.out.println("=== Fin de la ficha ===");
37
38  }
39 }

```

Lo anterior compila y se ejecuta de manera correcta, sin mostrar los comentarios agregados (como es de esperarse) y podemos verlo en la **Figura 8**.



```
30 de sep. 20:45
Escribir para buscar
guillermo@memo:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$ javac Programa
a.java
guillermo@memo:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$ java Programa
=== Ficha de compra ===
- Producto : Laptop para la carrera
- Precio con descuento : 12000
- Plazo de pago en años : 1.5
- Pago mensual : 666.6666666666666
=== Fin de la ficha ===
guillermo@memo:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$
```

Figura 8: *Compilación y ejecución de Programa.java* que muestra la información sin problema después de haber agregado los comentarios.

Ahora explicaremos brevemente por que creemos que es de utilidad cada comentario, según la enumeración de líneas, de lo escrito anteriormente:

- **Línea 1:** Para alguien que no entienda en su totalidad el programa, es indispensable agregar para que sirve la clase **Programa** y no solo escribir que es una simple declaración.
- **Línea 5:** Explicar para que funciona todo el método **main**, también es importante, pues indica que esta será la parte fundamental para que la máquina virtual de **Java** (**JVM**) ejecute el programa sin problemas.
- **Línea 8:** Dejamos en claro que la variable se declaro, para saber de que producto estamos hablando, en este caso, una laptop.
- **Línea 11:** Indicamos precio del producto, no solo ponemos que es una simple declaración, si no porque es importante la información que le asignamos.
- **Línea 14 y 17:** Mismo caso que en la **Línea 11**.
- **Línea 20:** Indicamos que texto se imprimirá en la pantalla con este **System.out.println**.
- **Línea 23:** Por la **Línea 20**, se puede concluir que se va a mostrar en la terminal, el texto dentro de “ ”, pero agregamos que se imprimirá la variable **producto**, además del texto.
- **Línea 26:** A parte de imprimir un texto, dejamos claro que realizara una operación e imprimirá dicha operación.
- **Lineas 29, 32 y 35:** Solo indicamos que se imprimirá lo que esta dentro del **System.out.println**, agregando en los casos donde se aplica, que se realizará una operación.

2.2.2. Actividad 2.2 Printf

Antes de comenzar con el desarrollo de esta sección, vamos a definir de manera breve el método `printf`.

`printf`

En **Java** el método `printf` se utiliza para imprimir texto y variables en la terminal. Es parecido a la ejecución que realiza `System.out.println`, solo que la forma en que se escribe la sintaxis para después mostrarlos en pantalla es distinta:

```
1 System.out.printf("Hello , %s!\n", "World");
```

Es decir, en vez de concatenar texto y variables con ayuda de "+", primero, escribe toda la cadena de texto:

```
1 "Hello , %s!\n"
```

Donde el `%s` indica que en ese punto específico del texto a imprimir, debe ir un **String**. ¿Cuál **String**?, el que colocamos después de una ",":

```
1 , "World"
```

La coma nos ayuda a indicar, que a partir de ahí, vamos a colocar el tipo de variable que queremos imprimir, según las variables con `%` que hayamos agregado entre comillas.

Dependiendo de que variable queremos agregar, después del `%`, escribimos cierta letra:

- `%d`: Sirve para indicar que imprimiremos un número entero **int**.
- `%f`: Sirve para indicar que imprimiremos un número flotante **float** o **double**.
- `%s`: Sirve para indicar que imprimiremos una cadena de texto **String**.
- `%c`: Sirve para indicar que imprimiremos un **char**.
- `%b`: Sirve para indicar que imprimiremos un valor de verdad **boolean**.

Precisión

La ventaja de un `printf`, es que este nos permite imprimir una variable en pantalla, con los decimales que nosotros le indiquemos, por ejemplo, imaginemos que tenemos 666,6666666666, si solo queremos que imprima tres dígitos después del punto decimal, con ayuda de `printf` debemos escribir:

```
1 System.out.printf("pago a meses: %.3f", 666.6666666666);
```

Lo cual nos imprimirá 666,667.

`printf` puesto en práctica

Con la información anterior descrita, pasemos con lo siguiente:

1. Vamos a copiar el archivo `Programa.java`, en la misma carpeta que lo guardamos, pero le vamos a cambiar el nombre a `ProgramaNuevo.java`, esto con el fin de tener el mismo programa, pero ahora utilizaremos lo aprendido con `printf` para imprimir de manera más adecuada, aquellos valores que tiene demasiados decimales.

2. En **ProgramaNuevo.java**, tenemos lo siguiente:

```
1      public class ProgramaNuevo{
2
3      public static void main(String[] args) {
4
5
6          String producto = "Laptop para la carrera";
7          int precio = 15000;
8          int descuento = 3000;
9          double meses = 18.0;
10
11         System.out.println("=== Ficha de compra ===");
12         System.out.println("- Producto : " + producto);
13         System.out.println("- Precio con descuento : " + (precio -
14             descuento));
15         System.out.println("- Plazo de pago en años : " + (meses /
16             12.0));
17         System.out.println("- Pago mensual : " + ((precio - descuento)
18             / meses));
19         System.out.println("=== Fin de la ficha ===");
20     }
21 }
```

Tenemos que modificar:

```
1      System.out.println("- Pago mensual : ", + ((precio - descuento
2          ) / meses));
```

Para que el valor que se imprima, solo tenga (en nuestro caso), solo dos decimales, después del punto, para esto, reescribimos como sigue:

```
1      System.out.printf("- Pago mensual : %.2f\n", ((precio -
2          descuento) / meses));
```

El código completo podemos verlo escrito en la **Figura 9**.

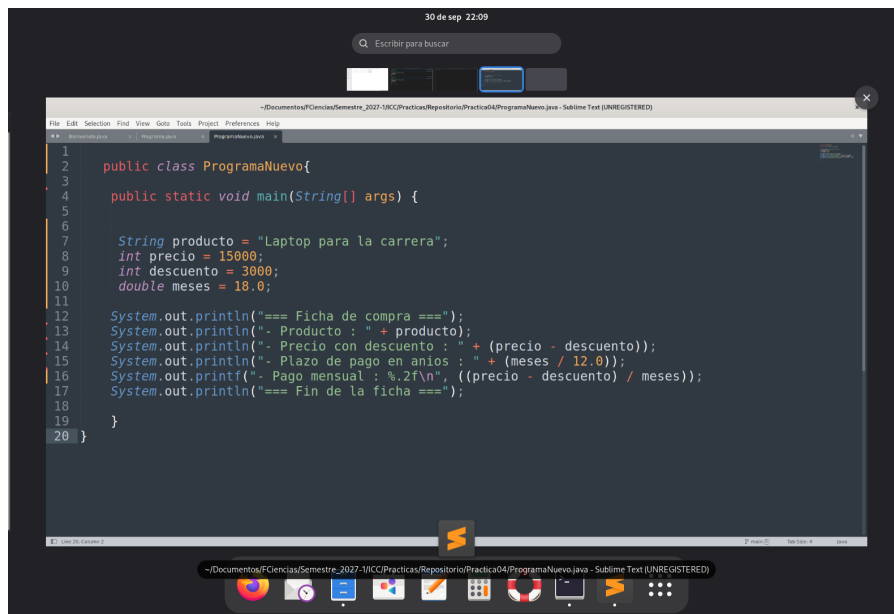


Figura 9: Código para la impresión de 2 dígitos de *pago mensual* en NuevoPrograma.java.

- Para esta última parte de `printf`, tenemos que imprimir toda la información anterior, pero en una sola línea de código, es decir, en vez de escribir todos los `System.out.println` que se ven en la **Figura 9**, solo queremos utilizar un `System.out.printf` para imprimir toda la información. Lo realizamos de la siguiente manera:

```
1      System.out.printf("=== Ficha de compra ===\n- Producto: %s\n-
    Precio con descuento: %d\n- Plazo de pago en años: %.1f\n-
    Pago mensual : %.2f\n=== Fin de la ficha ===\n", producto, (
    precio - descuento), (meses / 12.0), ((precio - descuento) /
    meses));
```

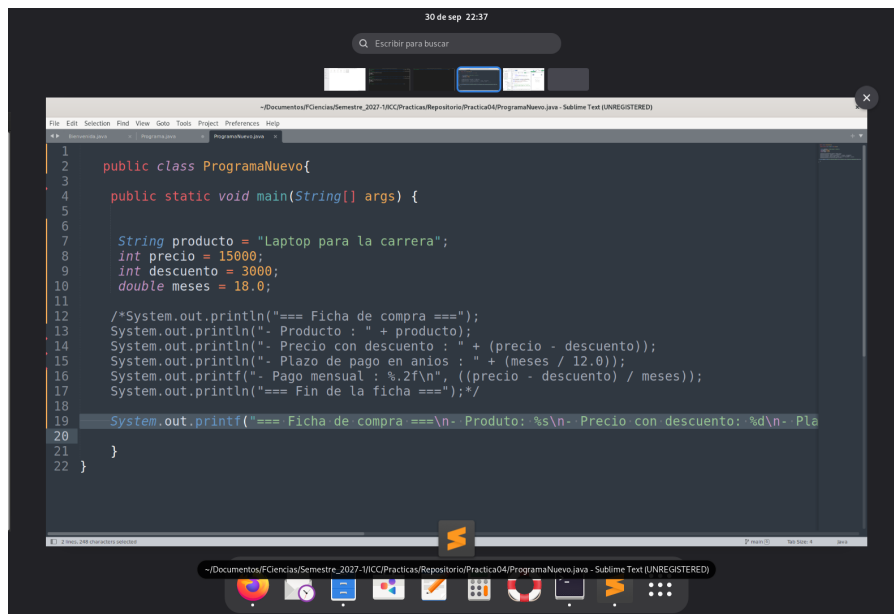
En cada parte donde correspondía un valor, hemos puesto `%s`, `%d`, `%.1f` y `%.2f` y respectivamente, después de la coma, colocamos:

```
1      , producto, (precio - descuento), (meses / 12.0), ((precio -
    descuento) / meses)
```

Es decir, cada `%`, separado con su correspondiente “,” es el valor que queremos imprimir:

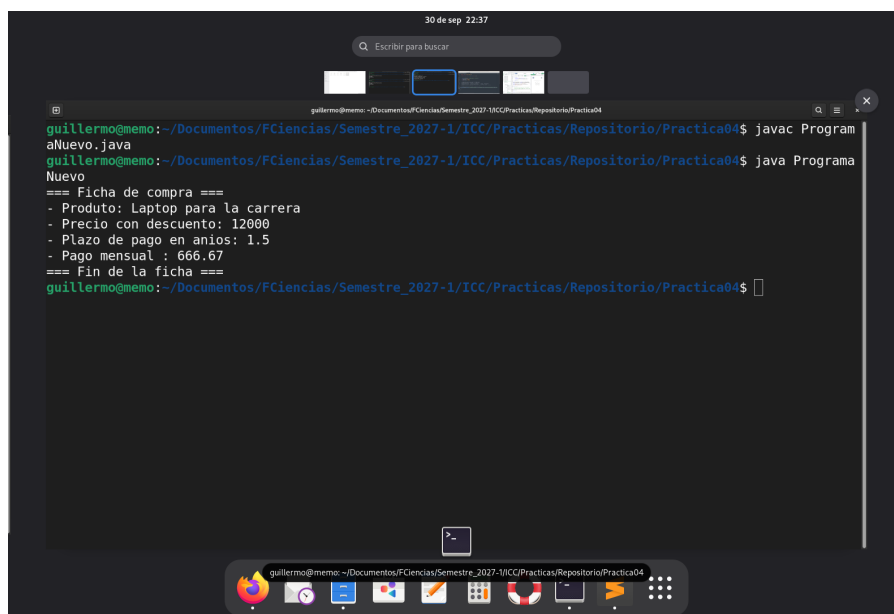
```
1      %s -> producto
2      %d -> (precio - descuento)
3      %.1f -> (meses / 12.0)
4      %.2f -> ((precio - descuento) / meses)
```

En la **Figura 10** observamos el código escrito para el programa y en la **Figura 11**, la compilación y ejecución del mismo.



```
1 public class ProgramaNuevo{
2
3
4     public static void main(String[] args) {
5
6
7         String producto = "Laptop para la carrera";
8         int precio = 15000;
9         int descuento = 3000;
10        double meses = 18.0;
11
12        /*System.out.println("=== Ficha de compra ===");
13        System.out.println("- Producto : " + producto);
14        System.out.println("- Precio con descuento : " + (precio - descuento));
15        System.out.println("- Plazo de pago en años : " + (meses / 12.0));
16        System.out.printf("- Pago mensual : %.2f\n", ((precio - descuento) / meses));
17        System.out.println("=== Fin de la ficha ===");*/
18
19        System.out.printf("=== Ficha de compra ===\n- Produto: %s\n- Precio con descuento: %d\n- Pla
20
21    }
22 }
```

Figura 10: Código para la impresión de toda la información de NuevoPrograma.java en un solo printf.



```
guillermo@memo: ~/Documentos/Fciencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$ javac ProgramaNuevo.java
guillermo@memo: ~/Documentos/Fciencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$ java ProgramaNuevo
=== Ficha de compra ===
- Produto: Laptop para la carrera
- Precio con descuento: 12000
- Plazo de pago en años: 1.5
- Pago mensual : 666.67
=== Fin de la ficha ===
guillermo@memo: ~/Documentos/Fciencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$
```

Figura 11: Ejecución y compilación de NuevoPrograma.java después de escribir todo en un solo printf.

2.3. Parte 3: Reto de abstracción

2.3.1. Actividad 3.1 - Filtrando información

Leyendo el problema planteado en el archivo `P4_Dandole_memoria_a_un_programa.pdf` correspondiente a esta actividad, vamos a clasificar los datos en necesarios e innecesarios:

Necesarios:

- **Plazo de meses a pagar:** Pensamos que es necesario puesto que este dato ayudará a calcular cuanto terminará pagando el cliente por lo que pidió prestado.
- **Total a pagar:** De igual manera, importante para saber que cantidad tendrá que pagar el cliente por lo que pidió prestado.
- **Mensualidades:** Importante para saber cuanto es lo que debe pagar por cada mes.
- **Nombre del cliente:** Dado que se necesita hacer una clasificación de si es cumplido o no a quien se le dio el préstamo,
- **Clasificación ‘P’ para los preferentes, ‘E’ para los estándar y ‘R’ para los riesgosos.:** El señor Pines necesita hacer esta clasificación, por lo que es importante tener en el programa, algo que detecte esta condición.
- **Precio de la Laptop:** A partir de el es que se desenvolverá prácticamente todo el programa.
Interés anual: Importante ya que sobre eso sabremos cuanto tendrá que pagar el cliente según la cantidad de dinero que se le preste.

Innecesarios:

- **Nombre del dueño:** No necesario para los cálculos del programa.
- **Nombre de la tienda:** Irrelevante de igual manera para obtener los datos finales del préstamo.
- **Novedad laptop para ICC:** A pesar de que es lo principal a comprar, no se necesita saber que es para realizar los cálculos.
- **Target de a quien se presta dinero:** no importa si presta a alumnos o personas regulares, al final del día el cálculo se hace sobre el dinero prestado.
- **RAM, SSD, Peso, Precio aproximado, Procesador, color de la laptop:** No necesitamos saber nada de estos aspectos, solo el precio final en conjunto del préstamo.

2.4. Actividad 3.2 - Cumpliendo los deseos del señor Pines

En este punto, hemos realizado el siguiente código, para el programa que pidió el señor Pines:

```
1 public class Cotizador{
2
3 public static void main (String [] args){
4
```

```

5  /*Declaracion de varibales necesarias, segun el caso particular del
   programa. No se declaro variable para el interes anual ni el dato 12
   meses, puesto que para este punto solo todos pagan 15 porciento
   anual sumado a que al ser anual, siempre se realiza el calculo con
   12 meses.
6
7  El estado del cumplimiento del cliente, lo colocamos en un string, sin
   mas, pues de nuevo
8  un solo estado para un solo cliente, despues podriamos usar un valor
   numerico con ayuda de algun if/else, que determine segun alguna
   condicion, que es cumplido, pero es trabajo a parte.
9  */
10     int precioCliente1 =12899;
11     String cliente1 = "Robbie Valentino";
12     String estadoCumplimientoCliente1 = "E";
13     int mesesPagoCliente1 = 21;
14     double intereses = precioCliente1*(15.0/100)*(mesesPagoCliente1
        /12.0);
15
16
17 //Detalle estetico
18     System.out.println("PRESTAMOS PINES");
19
20 // Estatus de cumplimiento del cliente
21     System.out.println("Estatus de "+ cliente1 + ": Cliente " +
        estadoCumplimientoCliente1);
22
23 //Pago de intereses
24     System.out.printf("\nPago de intereses al final del plazo:
        %.2f\n", intereses);
25
26 /*Total de monto a pagar, al ser un solo cliente, puede calcularse en
   el mismo printf para futuro podriamos crear una variable que guarde
   los precios totales que cada cliente paga para imprimir mas facil,
   pero de nuevo, es un trabajo a parte.
27 */
28     System.out.printf("Total de monto a pagar: %.2f\n",
        precioCliente1+intereses);
29
30 //mismo caso que el printf anterior
31     System.out.printf("Pago por meses: %.2f\n", (precioCliente1
        +intereses)/mesesPagoCliente1);
32
33
34 /*El programa puede mejorarse, pero solo hasta que el cliente lo
   requierra*/
35
36
37 }
38
39
40 }

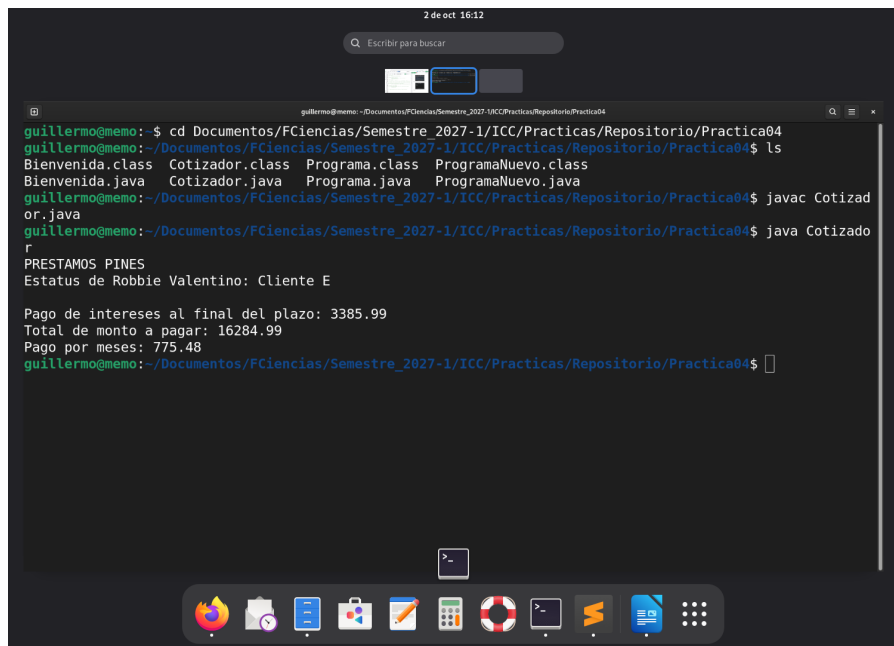
```

En cada punto anterior, hemos explicado con comentarios por cada decisión, pero a grandes

rasgos, realizamos un programa sencillo y solo con las cosas que sabemos y que nos pidieron, a pesar de que podemos claramente mejorar y agregar cosas que no solicitaron con el fin de entregar algo muy bueno, es preferible mantener el código simple y si después el cliente requiere alguna mejora, poder obtener algún beneficio por el trabajo extra, es decir, valorar más nuestro trabajo.

Con lo anterior, no justificamos el hecho de que es posible que algo se programe mal en el código del programa, solo del porque se tomaran las decisiones de lo que escribimos dentro del archivo y el como esta implementado.

En la **Figura 12**, observamos el código compilado y ejecutado.



```
2 de oct 16:12
Escribir para buscar

guillermo@memo: ~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04
guillermo@memo: $ cd Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04
guillermo@memo: ~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$ ls
Bienvenida.class  Cotizador.class  Programa.class  ProgramaNuevo.class
Bienvenida.java   Cotizador.java   Programa.java   ProgramaNuevo.java
guillermo@memo: ~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$ javac Cotizador.java
guillermo@memo: ~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$ java Cotizador
PRESTAMOS PINES
Estatus de Robbie Valentino: Cliente E

Pago de intereses al final del plazo: 3385.99
Total de monto a pagar: 16284.99
Pago por meses: 775.48
guillermo@memo: ~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio/Practica04$
```

Figura 12: *Ejecución y compilación del programa realizado para el señor Pines, es decir Cotizador.java.*

3. Conclusión

La realización de la práctica nos ayudo a poder entender, la declaración de variables mediante la experimentación con los tipos, las asignaciones y los nombres que colocábamos, concluyendo mediante práctica y no solo con lo visto en teoría, que puede asignarse a una variable o que no.

Implementamos comentarios dentro del código, entendiendo que en muchos casos, aunque puede parecer algo “innecesario”, en la realidad, por muy claro que uno pueda ser con los nombres de las variables y lo que se intenta imprimir, el contexto de comentar algo, puede ayudar a disipar ideas erróneas a la persona que intenta leer nuestro código por razones correctas (como los es, colaborar con nosotros, en el desarrollo de la programación), comprendiendo que aun cuando la cantidad del texto del comentario sea mucha, esto no afecta a la compilación y ejecución del programa.

Entendimos, los conocimientos actuales, aunque sólidos, en ocasiones pueden quedarse cortos cuando intentamos resolver un problema, en este caso aprendimos lo que es `printf` y su utilidad para la impresión de información en la terminal y los beneficios de trabajar con el, facilitando un poco la sintaxis al programar.

Finalmente, con todas las herramientas obtenidas, identificamos las cosas necesarias que pueden solicitarnos al resolver un problema, para descartar todo aquello que no aporta nada a nuestra programación y desarrollo de código, dejando de la manera más “ eficiente” la información vital para el correcto funcionamiento del programa.